Data analysis in the shell with **dftk** :: CHEATSHEET

One command per step, joined by pipes: `dftk CMD [FILE] [options]`. TSV and parquet in; TSV, parquet, JSON, Markdown or LaTeX out. Statistics with pandas, scipy and statsmodels; SQL with DuckDB; plots with matplotlib.

Basics

`dftk CMD [FILE] [options]` every command reads TSV or parquet and writes TSV to stdout

`cat data.tsv | dftk stat -c x` no FILE: read stdin

`dftk dataset tips -o | dftk stat -c tip` `-o` alone writes a temp parquet and prints its path; the next command reads it (types kept, fast, deleted after use)

`dftk CMD -h` options and examples, ready to run

`dftk dataset --list` about 50 example tables: tips, iris, penguins, diamonds, …

`dftk skill install` teach Claude Code and Codex to use dftk

Examples below leave out their input: run them as `dftk dataset tips -o | dftk . . .` *(or iris, penguins, diamonds, as the columns show).*

Read: options of every command

`--prequery "tip > 5"` filter rows as they are read

`--noheader` no header row: columns are V1, V2, …

`--delimiter ,` CSV or other separators

`--readasobject id` keep a column as text (`007` stays `007`)

`grep -v '^#' f.tsv | dftk CMD` drop comment lines first

`iconv -f latin1 -t utf-8 f.txt | dftk CMD` Latin-1 file; `sed -E 's/([0-9]),([0-9])/\1\2/g'` drops thousands commas

`dftk query --sql "SELECT * FROM 'f.jsonl'"` JSON and JSON Lines through DuckDB (no input needed); `pos.*` flattens, `unnest(tags)` explodes

Write: options of every command

`-o out.tsv` to a file; `.parquet`, `.json`, `.jsonl`, `.md` (table) and `.tex` (booktabs tabular) by extension

`--format json` the same formats to stdout: tsv json jsonl md tex

`--select day 'tot*' +` choose and order columns; globs; `+` keeps the rest

`--drop size --move tip:gratuity` drop; rename (OLD:NEW)

`--sortasc day tip` sort; `--sortdesc` descending; no names: every column

`--sort day:a tip:d` mixed directions

`--round tip:1 --digits 3` decimals; significant digits

`--cast size:Int64` change a column's type

`--deduplicate day --dropna` unique rows (by columns); drop rows with gaps

`--postquery "tip > 5"` filter the output

`--meta source=run7` provenance stored in parquet output

Inspect and filter

`dftk info` types, missing values, memory per column

`dftk describe` profile with quality flags

`dftk stat -c tip total_bill -g day` n, mean, std, median, CI, sem, skew … per group

`dftk query -q "tip > 5 and day == 'Sat'"` keep rows (pandas expression)

`dftk query --sql "SELECT day, avg(tip) AS tip FROM data GROUP BY day"` any DuckDB SQL; the input is the table `data`

`dftk sample -n 5 -g day --randomseed 1` random rows, per group

`dftk split -g day -s .tsv` one file per group: split-Sun.tsv …

`dftk cleannames` `Sample ID` → `sample_id`, `GC %` → `gc_pct`

`dftk print data.parquet` any input as TSV (or `--format`)

`dftk transpose` rows become columns

New columns

eval: formulas

`dftk eval -f "rate = tip / total_bill"` pandas expression

`dftk eval -f "m = mean(sepal*)"` row sum, mean, std, min, max, median over columns (globs)

`dftk eval -f "big = total_bill > 30" -f "size_txt = where(big, 'large', 'small')"` conditional value

`dftk eval -f "tip = tip.fillna(0)"` fill gaps; `.ffill()`, `.interpolate()`

`dftk eval -f "d = tip.groupby(day).diff()"` change from the row above, per group

`dftk eval -c "batch = 3" -c 'code = "007"'` constants (quoted: text)

`dftk eval -s "n = len(day)"` string, path, DNA and hash functions

Also: `fisher_test` `binom_test` `t2pval` `pval2se` … (`dftk eval -h`)

Other column builders

`dftk func -c tip -g day -t mean` group mean on every row; sum count rank pct_rank cumsum `qcut:4`

`dftk binx -c total_bill -b 0:60:10` bin index (or `--usevalue m`: bin middle)

`dftk scale -c tip -g day` z-score; `--rank` normal scores; `--resid -f "tip ~ total_bill"`

`dftk segid -c day` id that steps up when the value changes

`dftk interp samples.tsv --ref curve.tsv -x x -v y` y at x from a reference curve

`dftk randvar --dist norm -d noise` random variates (`-n 100`: no input)

SQL with DuckDB

`dftk query --sql "..."`: the input is the table `data`; a file name in quotes is a table too.

`SELECT * EXCLUDE (sex, smoker) FROM data` all columns but some

`SELECT day, count(*) AS n, avg(tip) AS tip FROM data GROUP BY ALL ORDER BY ALL` summary per group

`SELECT * FROM data QUALIFY row_number() OVER (PARTITION BY day ORDER BY tip DESC) <= 2` top 2 rows per group

`SELECT *, tip - lag(tip) OVER (PARTITION BY day ORDER BY total_bill) AS dtip FROM data` compare with the row above, per group

`SELECT * FROM data JOIN 'labels.tsv' USING (day)` join a file

`SELECT * FROM data USING SAMPLE 10` random rows

Combine and reshape

`dftk merge left.tsv -r right.tsv -k id -t left` join on key columns; inner left right outer cross

`dftk merge left.tsv -r right.tsv -k id --only left` rows only in left (anti-join)

`dftk concat a.tsv b.tsv --sourcecol src` stack tables; columns matched by name

id	a	b
1	10	20
2	30	40

melt →
← pivot

id	variable	value
1	a	10
2	a	30
1	b	20
2	b	40

`dftk melt -i species` wide → long: variable, value

`dftk pivot -i day -g time -v tip -f mean` long → wide, with aggregation (columns `mean_tip_Lunch` …); `-z` fills missing cells

`dftk explode -c go --sep ,` `GO:1,GO:2` → one row each; `implode` joins them back

`dftk append -c sample=S1 -c reads=48213` one row; `--format json` makes a JSON record

Statistics

`dftk stat -c tip -g day --bootstrap 1000 --randomseed 1` bootstrap CI of the summary

`dftk wstat -c total_bill -w size -g day` weighted mean, std, quantile CI

`dftk test -c tip:total_bill -d p --test mannwhitneyu` p-value per pair: student_t paired_student_t wilcoxon kruskal anova kolmogorov_smirnov bootstrap …

`dftk fit -f "tip ~ total_bill + C(day)"` OLS (R formula); `-r` robust, `-w` WLS, `--anova`, `-g`

`dftk corr -c tip:total_bill --method spearman --ci` correlation, p, CI; `--bootstrap 1000`

`dftk corr -c '*:*'` every pair once; `'sepal*:petal*'` cross product

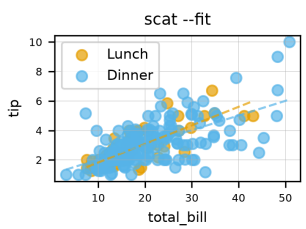
`dftk corr --all` square matrix for a correlogram (see Plots)

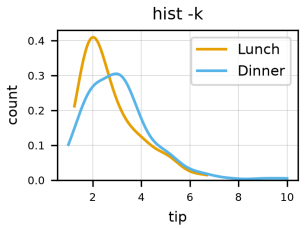
`dftk corr -c carat:price --tail` tail dependence; plot it with `chi`

`dftk pivot -v tip -i day -f mean --bootstrap 1000 --randomseed 1` bootstrap CI of grouped means

p-values are not adjusted for multiple testing; `corr` drops missing values pair by pair.

Plots

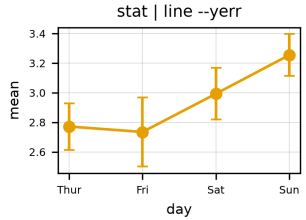


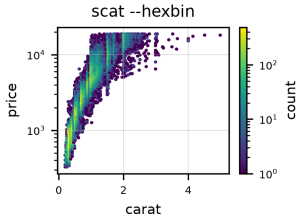


`dftk scat -x total_bill -y tip -g time --fit` scatter, OLS line per group; `--fit lowess`; `--hexbin` for big tables

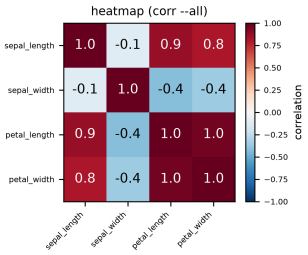
`dftk hist -x tip -g time -k` histogram, `-k` KDE; `-b 20`, `-n` density, `-c` cumulative

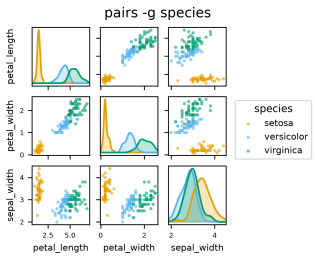
`dftk line -x day -y mean --yerr sem` lines with error bars; `--yci cilo,cihi`





`dftk scat -x carat -y price --hexbin --hexlog -ly` hexagons instead of 54 000 points





`dftk heatmap -x col1 -y col2 -v correlation --annot --vmin -1 --vmax 1` coloured grid; after `corr --all -o`: a correlogram

`dftk pairs -g species --upper corr` scatter-plot matrix, KDE diagonal; r overall and per group

`dftk chi -x carat -y price` chi-plot: lower vs upper tail dependence

Figure options

`-f fig.png` (or .pdf .svg; no `-f`: a window) • `--size double` or 6x4 • `-t TITLE` • `-xl -yl` labels • `-lx -ly` log axes • `--subgraphcol COL` panels • `--groupcolorder` • `--fontsize presentation`

Pipelines

Summarise, then plot

```
dftk dataset tips -o \
  | dftk stat -c tip -g day -o \
  | dftk line -x day -y mean --yerr sem -f tip.png
```

Correlogram

```
dftk dataset iris -o | dftk corr --all -o \
  | dftk heatmap -x col1 -y col2 -v correlation \
      --annot --vmin -1 --vmax 1 -f corr.png
```

SQL, then a table for a paper

```
dftk dataset penguins -o | dftk query --sql \
  "SELECT species, sex, avg(body_mass_g) AS mass
   FROM data WHERE sex IS NOT NULL
   GROUP BY ALL ORDER BY ALL" --digits 4 -o mass.tex
```

Status log for a workflow

```
dftk append runs.tsv --inplace -c step=align \
  -c status=ok --timestamp
```

Creates the file if missing; safe for parallel jobs; `.jsonl` and `.parquet` too; `--strict` refuses new columns.

In a Snakemake rule

```
rule summary:
    input: "data/{s}.tsv"
    output: "results/{s}.stats.tsv"
    shell: "dftk stat {input} -c x -g grp -o {output}"
```

More patterns: `references/snakemake.md` in `dftk skill path`.

Tips

- Quote globs and expressions: `'*:*'`, `"x > 0"`.
- Give FILE first: `dftk print f.tsv --sortasc`.
- `set -o pipefail` in scripts (Snakemake does).
- A temp pipe file is deleted when read; `-o name.parquet` keeps one.
- `dftk annotate f.parquet --set genome=hg38` stores notes in parquet.

Example data: dftk dataset NAME

tips (244 rows) `total_bill tip sex smoker day time size`

iris (150) `sepal_length sepal_width petal_length petal_width species`

penguins (344) `species island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g sex`

diamonds (53 940) `carat cut color clarity depth table price x y z`